FundooNotes Application Guide (Hinglish)

Complete Project Structure, Architecture Analysis,

aur Har Ek File ka Hinglish Explanation

Developer Documentation (Hinglish)

Bridgelabz Training - Day 16

Generated by Antigravity Coding Assistant

1. Project Architecture & Purpose (Project Kya aur Kyu?)

FundooNotes ek clean, secure aur enterprise-grade notes management application hai jise Spring Boot framework par banaya gaya hai. Ye Google Keep ka clone hai, jismein har user apne notes ko create, update, delete, search, archive, trash aur restore kar sakta hai. Is project ko scalability aur industry standard ke rules ke according banane ke liye multiple advanced tools (jaise Caching aur Message Queues) integrate kiye gaye hain.

Hum is project mein kya aur kyu kar rahe hain?

Modern systems mein high response speed, background tasks management, security aur code structure bahut key components hote hain. Is project ke through humne in primary patterns ko build aur master kiya hai:

*** Spring Security & JWT:** APIs ko fully secure rakhne ke liye taaki bina permission ke koi user kisi doosre user ke notes read ya write na kar sake. Dynamic stateless authentication standard follow karta hai.

*** Redis Caching:** Read response time ko reduce karne ke liye notes search list aur repetitive cryptographic token validations ko Redis in-memory storage mein temporary cache kiya jata hai.

*** RabbitMQ (AMQP):** Asynchronous events logging ke liye. Jab bhi koi user note par delete, create ya archive perform karega, to response block kiye bina background queue mein logging process event chala jayega.

*** ActiveMQ (JMS):** Background workflows management ke liye. Jaise forgot password scenario mein mail notification generation task queue ko dispatch hota hai, jahan background consumer bina normal user execution hold kiye background flow run karta hai.

*** MySQL & Spring Data JPA:** Application ka transactional relational database records clean state manage karne ke liye.

Project Architecture Structure Design

Is project mein clear Layered Architecture ka use kiya gaya hai:
- Controller Layer: Requests receive karta hai aur direct standard DTO response serialize karta hai.
- Service Layer: Poori logical checks, caching annotations implementation aur message queue triggers handle karta hai.
- Repository Layer: JPA queries run karke database connectivity support deta hai.
- Messaging Layer: Decoupled systems components (RabbitMQ aur ActiveMQ) coordinate karta hai.
- Security Layer: Requests filter check chain apply karta hai.

2. Core Functionalities Implemented (Features ki List)

*** User Authentication Flow:** Secure register process (BCrypt double encryption security) aur login par secure cryptographically signed JWT token response.

*** Forgot & Reset Password:** Email entry validation par UUID recovery key creation jo 15 mins tak valid hoti hai. Recovery logs activeMQ queue mein async forward ho kar listener side simulation run karte hain.

*** Secure Note Operations:** Har query database run hone se pehle current user access mapping matching checks apply karti hai.

*** Archive & Trash Status Handling:** Note state transition (ACTIVE, ARCHIVED, TRASHED) and trash notes safety restrictions (Trash wale notes pin nahi ho sakte).

*** Note Pinning:** Notes ko search listings mein dynamically key rules ke top priorities status maps update.

*** Dynamic Filtering (JPA Specification):** Criteria builder dynamically query forms combine karta hai jaise Title matches, note state checks, aur pinned conditions.

*** Tag and Labels Management:** ManyToMany join integration schema structure handle linking details.

*** Redis Cache eviction rules:** Search outputs `@Cacheable` se store hote hain aur dynamic modify checks par `@CacheEvict(allEntries=true)` se clear aur refresh manage hote hain.

*** Audit logging pipelines:** Create/Delete triggers async format event publish to RabbitMQ.

3. Project Structure & Packages Directory

Hamara project package structure standard Spring layers design structure control karta hai:

- `config`: Security components, database serialization settings aur queue connections variables configure hote hain.
- `controller`: REST APIs endpoints mapping handle actions mappings.
- `dto.request`: Request attributes data validation rules standard models.
- `dto.response`: Client formatted clean data serialization structures.
- `entity`: Relational schema structures (User, Note, Tag, PasswordResetToken).
- `mapper`: Entity models ko DTO maps data flow transfer helper converters.
- `messaging`: ActiveMQ/RabbitMQ components variables, queues mappings, producers aur consumers listeners.
- `repository`: Standard JpaRepository database queries interfaces mappings.
- `security`: Custom filter chains, Token parsing engines, security context integrations.
- `service`: Core dynamic execution signatures contracts interfaces.
- `service.impl`: Class algorithms implementation, database calls checks aur caching variables integration.
- `specification`: Dynamic SQL criteria builder formulas helper predicates code structures.
- `resources`: Credentials configuration aur database properties variables.

4. File-by-File Walkthrough & Code Explanations

APPLICATION MAIN ENGINE

FundooNotesAppApplication.java

Ye project ka core entry bootstrap class hai. Yahan `@SpringBootApplication` ke sath do essential annotations: `@EnableCaching` (Redis runtime processing activate karne ke liye) aur `@EnableJms` (ActiveMQ background listener engine active karne ke liye) use kiye gaye hain.

```
@SpringBootApplication
@EnableCaching
@EnableJms
public class FundooNotesAppApplication {
    public static void main(String[] args) {
        SpringApplication.run(FundooNotesAppApplication.class, args);
    }
}
```

RABBITMQ CONFIGURATION SETTINGS

config/MessagingConfig.java

Is configuration class mein RabbitMQ queue configuration maps banate hain. Topic Exchange 'note-exchange' build hota hai, Queue name 'note-audit-queue' declare hota hai aur 'note.audit.routingkey' key ke through bind

rules mapping register hoti hai.

```
public static final String QUEUE = "note-audit-queue";
public static final String EXCHANGE = "note-exchange";
public static final String ROUTING_KEY = "note.audit.routingkey";
@Bean
public Queue queue() { return new Queue(QUEUE, true); }
@Bean
public TopicExchange exchange() { return new TopicExchange(EXCHANGE); }
@Bean
public Binding binding(Queue queue, TopicExchange exchange) {
    return BindingBuilder.bind(queue).to(exchange).with(ROUTING_KEY);
}
```

REDIS CACHE SETUP

config/RedisConfig.java

Ye class application cache engine rules register karta hai. JSON formatting conversion map support ke liye `GenericJackson2JsonRedisSerializer` register kiya gaya hai taaki object data JSON format store ho sake. default cache validity expiration rule (TTL) ko 10 minutes par set kiya hai.

```
@Bean
public RedisCacheManager cacheManager(RedisConnectionFactory connectionFactory) {
    RedisCacheConfiguration config = RedisCacheConfiguration.defaultCacheConfig()
            .entryTtl(Duration.ofMinutes(10))
            .disableCachingNullValues()
            .serializeKeysWith(...StringRedisSerializer...)
            .serializeValuesWith(...GenericJackson2JsonRedisSerializer...);
    return RedisCacheManager.builder(connectionFactory).cacheDefaults(config).build();
}
```

SPRING SECURITY RULES MANAGER

config/SecurityConfig.java

Ye file security filter routes manage karti hai. CSRF security parameters ko stateless design pattern (JWT use karne par) ke karan disable rakhte hain. Public pages '/auth/register', '/auth/login', '/auth/forgot-password' aur '/auth/reset-password' ko bypass (permit all) karke bache hue secure links par custom `JwtAuthenticationFilter` rule check active karta hai.

```
http.csrf(csrf -> csrf.disable())
    .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
    .authorizeHttpRequests(auth -> auth
        .requestMatchers("/auth/register", "/auth/login", "/auth/forgot-password", "/auth/reset-password").permitAl
        .anyRequest().authenticated()
    )
    .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);
```

JWT ENCRYPTION AUR VALIDATION HELPER

security/JwtUtil.java

Is utility mein token generation aur validation algorithm codes hain. Token verification database check performance ko improve karne ke liye token verification output true state ko Redis in-memory storage mein key `jwt:valid:<token>` format value 'true' 10 mins store settings maps run karta hai taaki direct fast responses verify ho sakein.

```
public String generateToken(String userId, String email) {
    return Jwts.builder().subject(userId).claim("email", email)
            .issuedAt(new Date()).expiration(new Date(System.currentTimeMillis() + 3600000))
            .signWith(getSigningKey()).compact();
}
public boolean isTokenValid(String token) {
    String cacheKey = "jwt:valid:" + token;
    String cached = redisTemplate.opsForValue().get(cacheKey);
    if ("true".equals(cached)) return true;
    Jwts.parser().verifyWith(getSigningKey()).build().parseSignedClaims(token);
    redisTemplate.opsForValue().set(cacheKey, "true", Duration.ofMinutes(10));
    return true;
}
```

HTTP REQUESTS TOKEN CHECKER INTERCEPTOR

security/JwtAuthenticationFilter.java

Ye custom filter check rules application request pipeline mein har client request run rules trace par access header verify karta hai. Header 'Authorization: Bearer <token>' extract logic lagata hai aur credentials validity valid checks milne par secure session contexts update rules setup execution handle karta hai.

```
String authHeader = request.getHeader("Authorization");
if (authHeader != null && authHeader.startsWith("Bearer ")) {
    String token = authHeader.substring(7);
    if (jwtUtil.isTokenValid(token)) {
        String userId = jwtUtil.extractUserId(token);
        String email = jwtUtil.extractEmail(token);
        CustomUserDetails userDetails = new CustomUserDetails(Integer.parseInt(userId), email, null);
        UsernamePasswordAuthenticationToken authentication = new UsernamePasswordAuthenticationToken(
                userDetails, null, userDetails.getAuthorities());
        SecurityContextHolder.getContext().setAuthentication(authentication);
    }
}
filterChain.doFilter(request, response);
```

SPRING SECURITY SESSION USER MODEL

security/CustomUserDetails.java

UserDetails interface implementer. Hamare custom parameters values logic mappings runtime execution sessions principal details store variables holding structure.

DATABASE USER MODEL SCHEMA

entity/User.java

JPA Model definition structure for `users` database table details. Fields contains database definitions: `userId` key auto increment, unique user `email` index parameter aur crypt BCrypt encoded details `passwordHash`. OneToMany relationship mappings for Notes setup automatic child entities cleanup rule (`orphanRemoval = true`).

```
@Entity
@Table(name = "users")
public class User {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int userId;
    @Column(nullable = false, unique = true) private String email;
    @Column(nullable = false) private String passwordHash;
    private String name;
    @OneToMany(mappedBy = "owner", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Note> notes = new ArrayList<>();
}
```

DATABASE NOTES MODEL SCHEMA

entity/Note.java

JPA Model mapping table coordinates for `notes`. Field specifications maps key variables title strings, content block size allocations maximum length limit definitions maps 2000 settings. Note current state options lifecycle mappings: Active state representation, Archived parameters filter matches status checks, and Trashed flag. Tag details map coordinates ManyToMany link mappings table.

```
public enum NoteState { ACTIVE, ARCHIVED, TRASHED }
@Entity @Table(name = "notes")
public class Note {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private int noteId;
    @Column(nullable = false) private String title;
    @Column(length = 2000) private String content;
    private LocalDateTime createdAt = LocalDateTime.now();
    @ManyToOne @JoinColumn(name = "user_id") private User owner;
    @ManyToMany
    @JoinTable(name = "note_tags",
               joinColumns = @JoinColumn(name = "note_id"),
               inverseJoinColumns = @JoinColumn(name = "tag_id"))
    private Set<Tag> tags = new HashSet<>();
    @Enumerated(EnumType.STRING) private NoteState state = NoteState.ACTIVE;
    private boolean pinned = false;
}
```

DATABASE LABELS MODEL SCHEMA

entity/Tag.java

Database mapping maps columns key attributes `tagId` index keys definitions parameters unique `name` specifications maps.

DATABASE RECOVERIES SCHEMA

entity/PasswordResetToken.java

Database table settings maps recovery tokens verification records, links keys `token` details, duration mapping expiry variables checks logic configurations.

AUTHENTICATION API CONTROLLER ROUTES

controller/AuthController.java

Public login routes controller endpoints mappings. Paths handle registers calls, login parameter checks verify validations, forgot triggers password recovery workflows triggers tokens aur reset parameters map requests updating properties codes.

```
@PostMapping("/register")
public ResponseEntity<AuthResponse> register(@Valid @RequestBody RegisterRequest r) {
    String token = userService.register(r);
    return ResponseEntity.status(HttpStatus.CREATED).body(new AuthResponse(token));
}
@PostMapping("/login")
```

```
public ResponseEntity<AuthResponse> login(@Valid @RequestBody LoginRequest r) {
    String token = userService.login(r);
    return ResponseEntity.ok(new AuthResponse(token));
}
```

NOTES CRUD ROUTING ENDPOINTS

controller/NoteController.java

Secure notes operations endpoints interfaces controllers maps details. Requests checks user validation session contexts, invokes search algorithms conditional lists parsing parameters options (title filters strings, archive states variables check).

```
private int currentUserId() {
    CustomUserDetails u = (CustomUserDetails) SecurityContextHolder.getContext()
            .getAuthentication().getPrincipal();
    return u.getUserId();
}
@GetMapping
public List<NoteResponseDTO> getMyNotes(@RequestParam(required = false) String title,
                                        @RequestParam(required = false) NoteState state,
                                        @RequestParam(required = false) Boolean pinned) {
    return noteService.searchNotes(currentUserId(), title, state, pinned);
}
```

TAGS API ROUTES CONTROLLER

controller/TagController.java

APIs routing handle creation, listings parameters search, deletions mapping configurations.

USER OPERATIONS LOGIC EXECUTION

service/impl/UserServiceImpl.java

Is implementation file mein primary logic check rules aur execution maps configure hain:
- Register: Throws dynamic exception errors is user matching email already exists, hashes input using BCrypt.
- Login: Authenticate matches dynamic encryption maps codes.
- Forgot Password: Builds UUID string dynamic recoveries key, commits metadata configurations to databases parameters reset entities maps, calls JMS message generation templates dispatch.

```
public String forgotPassword(String email) {
    User user = userRepository.findByEmail(email).orElseThrow(() -> ...);
    String token = UUID.randomUUID().toString();
    PasswordResetToken resetToken = new PasswordResetToken();
    resetToken.setToken(token); resetToken.setUser(user);
    resetToken.setExpiryTime(LocalDateTime.now().plusMinutes(15));
    passwordResetTokenRepository.save(resetToken);
    reminderProducer.sendPasswordResetRequest(email, token); // ActiveMQ JMS Queue
    return token;
}
```

NOTES CORE BUSINESS EXECUTION LOGIC

service/impl/NoteServiceImpl.java

Is file mein notes CRUD aur updates parameters manage logics define kiye hain, jismein caching implementation dynamic check flows integrate hain:
- Caching rules: `searchNotes` lists data dynamic keys patterns format (`#userId + '_' + ...`) use karke Redis cache block storage mapping save logic implement karta hai.
- Cache eviction logic: Mutate action calls triggers (createNote, archive, pin actions) use annotations `@CacheEvict(allEntries=true)` to remove old search lists keys parameters validation configurations cached sets.
- Logging: RabbitMQ audit operations triggers.

```
@Override
@CacheEvict(value = "notesList", allEntries = true)
public NoteResponseDTO createNote(int userId, NoteRequestDTO request) {
    User owner = userRepository.findById(userId).orElseThrow(...);
    Note note = new Note(); note.setTitle(request.getTitle());
    note.setContent(request.getContent()); note.setOwner(owner);
    Note savedNote = noteRepository.save(note);
    auditProducer.sendAuditLog("CREATE", (long) savedNote.getNoteId(), ...);
    return noteMapper.toResponseDTO(savedNote);
}
@Override
@Cacheable(value = "notesList", key = "#userId + '_' + ...")
public List<NoteResponseDTO> searchNotes(int userId, String title, NoteState state, Boolean pinned) {
    User owner = userRepository.findById(userId).orElseThrow(...);
    Specification<Note> spec = NoteSpecification.hasOwner(owner);
    if (title != null && !title.isBlank()) spec = spec.and(NoteSpecification.hasTitle(title));
    if (state != null) spec = spec.and(NoteSpecification.hasState(state));
    if (pinned != null) spec = spec.and(NoteSpecification.isPinned(pinned));
    return noteRepository.findAll(spec).stream().map(noteMapper::toResponseDTO).toList();
}
```

TAG BUSINESS LOGIC

service/impl/TagServiceImpl.java

Tag criteria checks unique name rules checks validations records update database entries registers.

DYNAMIC SQL QUERIES BUILDERS

specification/NoteSpecification.java

JPA Criteria Specifications use templates builders configurations map predicates dynamically, such as Owner validation parameters, Title LIKE matching patterns structures dynamic formulas SQL outputs mapping builders.

```
public static Specification<Note> hasTitle(String title) {
    return (root, query, cb) -> cb.like(cb.lower(root.get("title")), "%" + title.toLowerCase() + "%");
}
public static Specification<Note> hasState(Note.NoteState state) {
    return (root, query, cb) -> cb.equal(root.get("state"), state);
}
```

JMS ACTIVEMQ QUEUE DISPATCHER

messaging/ReminderProducer.java

JmsTemplate parameters triggers queue integrations sender utilities logic operations maps, packs parameters `email|token` as a raw payload string payload details structure and dispatches event trigger calls to `password-reset-queue` background queues.

```
public static final String PASSWORD_RESET_QUEUE = "password-reset-queue";
public void sendPasswordResetRequest(String email, String resetToken) {
    String payload = email + "|" + resetToken;
    jmsTemplate.convertAndSend(PASSWORD_RESET_QUEUE, payload);
}
```

ACTIVEMQ BACKGROUND JMS LISTENER QUEUE PROCESSOR

messaging/ReminderConsumer.java

ActiveMQ consumer logs. `@JmsListener` target path references `password-reset-queue`. Parses event contents splits key metrics, performs sleep simulations processing dispatch `Thread.sleep(3000)` mapping background simulator notification emails delivery execution.

```
@JmsListener(destination = ReminderProducer.PASSWORD_RESET_QUEUE)
public void consumePasswordResetRequest(String payload) {
    String[] parts = payload.split("\\|");
    if (parts.length == 2) {
        String email = parts[0]; String token = parts[1];
        Thread.sleep(3000); // 3 seconds mail sending delay simulation
        System.out.println("Password reset email sent to: " + email);
    }
}
```

RABBITMQ TOPIC EXCHANGE QUEUE PRODUCER

messaging/AuditProducer.java

RabbitMQ message writer engine. Serializes event messages using exchange coordinates and routing routingkeys matching rules parameters mappings.

```
public void sendAuditLog(String action, Long noteId, String details) {
    String message = String.format("Action: %s | Note ID: %d | Details: %s | Timestamp: %d",
            action, noteId, details, System.currentTimeMillis());
    rabbitTemplate.convertAndSend(MessagingConfig.EXCHANGE, MessagingConfig.ROUTING_KEY, message);
}
```

RABBITMQ EVENTS LISTENER CONSUMER LOGGERS

messaging/AuditConsumer.java

RabbitMQ listener registers `@RabbitListener(queues = MessagingConfig.QUEUE)`. Receives data logging operations records to output consoles prints.

```
@RabbitListener(queues = MessagingConfig.QUEUE)
```

```
public void consumeAuditLog(String message) {
    System.out.println("RabbitMQ Audit Consumer received message: " + message);
}
```

SYSTEM MODELS MAPS DATA CLASSES

Mappers & Data Transfer Objects (DTOs) Summary

Lombok `@Data` request responses models separate clean data APIs attributes from core JPA records entities models columns:
- `RegisterRequest`/`LoginRequest`: validation attributes inputs checks controllers models.
- `NoteRequestDTO`/`NoteResponseDTO`/`TagResponseDTO`: Clean serializable properties properties templates.
- `NoteMapper`/`TagMapper`: standard mapping logic helper converters methods maps entities variables properties stream patterns.